CHARUSAT

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 1:

Array Operations: Traversal, Rotation, and Frequency Analysis

Practical Aim - To implement fundamental array operations.

Problem 1 :

A bakery prepares n items every morning and places them in a display row. At the end of each hour, the leftmost item is moved to the rightmost position to make room for fresh stock at the front. Given the initial row and the number of hours h, print the final display order. Describe the approach you used. How does it behave when h is very large, say 10 million? Is there a way to get the correct result without performing the operation h times?

CODE-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical1/1_1.cpp

OUTPUT:



Problem Faced-

During this Practical ,The problem I faced First was First thinking the Logic only Otherwise the Problem was easy Once understood...

Conclusion

The program successfully performs the left rotation of the array by h positions. Using h % n helps reduce unnecessary rotations when h is very large, making the solution more efficient.

Problem 2:

A library issues books to students and records each book's ID every time it is borrowed. At the end of the month, the librarian wants to find all books that were borrowed more than once, as those need priority restocking. Given the borrowing log, print all such book IDs. Describe the approach you used. How many times does your solution scan the data? If the library had 100,000 borrow records, would your approach still be practical? Can you think of a way that requires fewer passes?

CODE-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical1/1_2.cpp

Output –



Problem Faced-

The problem Faced during the execution of the Practical was to return the no that was duplicated ..

Conclusion

The program successfully finds the book IDs that were borrowed more than once. This helped me understand how to identify duplicates in an array and also made me think about how using a better approach, like a frequency map, can reduce the number of scans and improve efficiency for large data.

Problem 3:

A school newspaper editor wants to highlight the most impressive word from a submitted article on the front page. The word is chosen simply by length — the longest one wins. Given a sentence, print the winning word and how many letters it has. Describe the approach you used. What does your solution do when two words have the same length? Is that behavior intentional, and can you think of a way to handle it explicitly?

CODE-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical1/1_3.cpp

Output-



Problem Faced

I initially found it a little difficult to compare the lengths of all the words and keep track of the longest one. I also had to make sure that the word and its length were updated correctly whenever a longer word was found.

Conclusion

The program successfully finds and prints the longest word along with its length. I learned how to process a sentence word by word and compare their lengths. If two

words have the same length, the current approach keeps the first one found, which is a simple and intentional choice.

KEY QUESTIONS-

1. [Problem 1] The display order repeats itself after a certain number of hours regardless of how large h gets. Without trying any specific example, explain why this repetition is inevitable and what determines when it first repeats.

   The display order repeats because after moving all n items once from left to right, every item comes back to its original position. So, the order repeats after every n hours. The value of n, which is the total number of items, determines when the first repetition occurs.

2. If the same book ID appears three times in the log, should it appear once or three times in your output? Does your solution handle this correctly, and how would you verify it?

It should appear **only once**, even if the same book ID appears three times. The solution should check whether an ID has already been added to the output. I would verify it by testing an input where one book ID occurs 3–4 times and checking that it is printed only once.

3. The editor later decides that punctuation attached to a word (e.g. “hello,” or “great!”) should not count toward its length. How would this change your solution, and at which step would you handle it?

I would remove the punctuation marks from each word before calculating its length. This would be handled while reading and processing each word, before comparing its length with the current longest word.

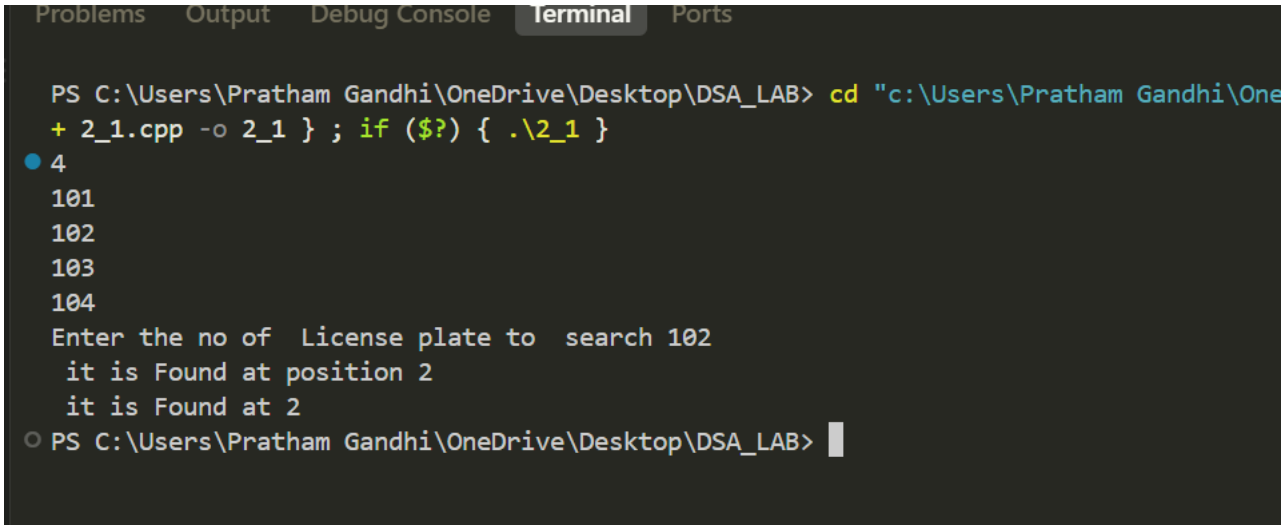
Practical 2: Linear and Binary Search: Iterative and Recursive Approaches

Practical Aim
To implement linear and binary search using both iterative and recursive approaches.

Problem 1 -
A security guard at a parking lot checks vehicles one by one from the entrance to find a car with a specific license plate. Sometimes he starts from the entrance, sometimes he calls a helper who starts from where the guard left off. Given a list of license plates and a target plate, implement both approaches — one that checks plates one by one from the start, and one where the function calls itself to continue checking — and report the position of the target plate if found. Describe the approach you used. What happens in your solution if the target plate appears more than once in the list? Does it find the first occurrence, the last, or just any one?

CODE-
https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical%202/2_1.cpp

OUTPUT-



Problem Faced-

The main difficulty was understanding how to implement the same search using both iteration and recursion. I had to make sure the recursive function stopped correctly when the target was found or when the list ended.

Conclusion

This problem helped me understand the difference between iterative and recursive searching. I learned how recursion can solve the same problem by calling the function again with the next position.

Problem 2-

A librarian maintains a sorted catalog of book codes and needs to locate a specific code quickly. Instead of going through every book, she opens the catalog to the middle, checks whether the target is to the left or right, and repeats. Given a sorted list of book codes and a target code, implement both approaches — one using a loop and one where the function calls itself — and report the position of the target code. Describe the approach you used. What is the one condition your input must satisfy for this approach to work correctly? What would go wrong if that condition was not met?

CODE-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical%202/2_2.cpp

Recursion Code-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical%202/2_2rec.cpp

Output-



Problem Faced

The main difficulty was understanding how to update the middle, left, and right positions correctly in both iterative and recursive approaches. I also had to make sure the target was searched in the correct half each time.

Conclusion-

This problem helped me understand binary search using both iteration and recursion. I learned that the input must be sorted for binary search to work correctly. If the data is not sorted, the search may move to the wrong half and give an incorrect result.

Key Questions

1 1. [Problem 1] If the target plate appears near the end of a very long list, your solution still checks every plate before it. Can you think of any property the input would need to have for you to skip checking some plates entirely? What does Problem 2.2 tell you about this?

If the license plates had some sorted or ordered property, we could skip checking many plates instead of checking each one. Problem 2.2 shows that binary search can do this when the data is sorted, reducing the number of checks significantly.

2 What is the consequence of applying your solution to a list that is not sorted? Would it always give a wrong answer, or only sometimes? Explain your reasoning without running any example.

It would give a **wrong answer sometimes**, not always. Binary search depends on the list being sorted to decide which half to search next. If the list is not sorted, that decision may lead to the wrong half, so the target could be missed even if it exists.

Practical 3:

Sorting Algorithms: Comparison-Based, Counting-Based, and Digit-Based Strategies

Practical Aim-

To implement and compare multiple sorting algorithms and develop the ability to reason about which sorting strategy suits a given situation.

Problem 1 -

A teacher has a stack of student answer sheets with marks written on them and needs to arrange them in order before entering grades. She tries three different methods: in the first, she repeatedly compares adjacent sheets and swaps them if they are out of order; in the second, she finds the lowest-marked sheet each time and places it at the front; in the third, she picks each sheet one by one and inserts it into its correct position among the already-arranged sheets.
Implement all three methods and for each one, print the sorted order of marks.
Describe how each of the three methods works in your own words. If the sheets were already in order before the teacher started, which method would finish the fastest and why?

Code-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical%203/3_1.cpp

Output-


it is Found at 2
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB> cd "c:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB\" ; if ($?)
+ 3_1.cpp -o 3_1 } ; if ($?) { .\3_1 }
5
1 23 44 24 65
Using Bubble Sort - 1 23 24 44 65
Using Selection Sort - 1 23 24 44 65
Using Insertion Sort - 1 23 24 44 65
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB>

Problem Faced-

The main difficulty was understanding the difference between the three sorting methods and implementing their swapping/insertion logic correctly. I also had to keep track of the positions carefully while sorting.

Conclusion

This problem helped me understand **Bubble Sort, Selection Sort, and Insertion Sort** in a practical way. I learned that if the marks are already sorted, **Insertion Sort** would usually finish fastest because it can detect that no shifting is needed and requires very little work.

Problem 2

A paint shop has buckets labeled with one of three colour codes — 0, 1, or 2 — but they are stored in a random order. The shop owner wants all 0s together, then all 1s, then all 2s, without using any extra storage. Given the list of colour codes, rearrange them in place and print the result. Describe the approach you used. Does your solution make more than one pass through the list? Can you think of a way to do it in exactly one pass?

Code-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical%203/3_2.cpp

Output-



Problem Faced

The main difficulty was rearranging the 0s, 1s, and 2s without using another array. I had to carefully manage the positions while swapping so that no element was missed.

Conclusion

This problem helped me understand how to sort elements **in place** using limited extra space. I also learned that the same problem can be solved in a single pass using the **Dutch National Flag algorithm**, making it more efficient.

KEY QUESTIONS-

1. [Problem 1] One of the three methods can detect early that the list is already sorted and stop without doing any more work. Which method is it, how would you add that detection, and why can the other two not do the same?

The method is **Bubble Sort**. We can add a swapped flag and set it to true whenever a swap happens. If no swap occurs in a complete pass, the list is already sorted, so we can stop early.

The other two methods don't naturally check the whole list for swaps in the same way, so they still continue their normal steps even if the list is already sorted.

2. If your solution makes two passes through the list, can you identify which decision in your logic forces the second pass? Is there a way to restructure that decision to eliminate it?

The second pass is needed because the solution first **counts the number of 0s and 1s**, and then uses another pass to place them in order. To make it one pass, we can use **three pointers** and place 0, 1, and 2 in their correct regions while scanning the list.

Practical 4:
Singly Linked List: Dynamic Insertion, Deletion, and Traversal

Practical Aim-

] To implement core singly linked list operations and develop the ability to manipulate dynamic sequences through real-world problem scenarios.

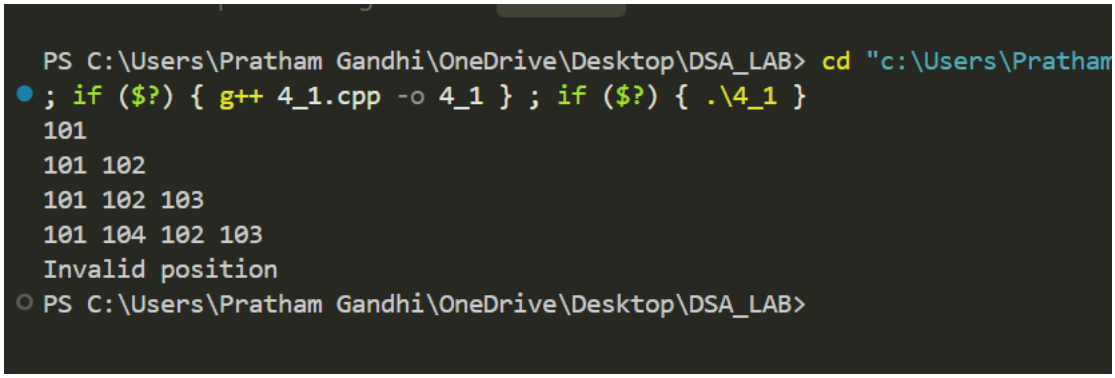
Problem 1 -

A hospital manages a queue of patient tokens. New critical patients must be added to the front, routine patients are added to the end, and occasionally a patient with a priority number must be inserted at a specific position in the queue. Given a sequence of such operations, implement all three insertion types and print the final queue after each operation. Describe how your insertion at a specific position works. What does your solution do if the position given is greater than the current length of the queue? Is that handled explicitly or does it cause an error?

Code-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical%204/4_1.cpp

Output-



Conclusion

In this practical we learnt how we can add and delete element from front ,end and at a given specific posiition and how we can print elements in linked list.

Problem 2

The hospital also needs to remove a patient token from the queue when a patient leaves, print all remaining tokens from last to first for an end-of-day audit, and display the full queue from front to back at any point. Implement deletion by value, reverse printing, and forward traversal on the same queue from Problem 4.1a. Describe how your reverse printing works. Does it use any extra storage, or does it work purely by following the links? Can you think of another way to reverse-print without modifying the original queue?

Code-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical%204/4_2.cpp

Output-

```
; if ($?) { g++ 4_2.cpp -o 4_2 } ; if ($?) { .\4_2 }
Forward: 101 102 103 104 105
After deletion: 101 102 104 105
Reverse: 105 104 102 101
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB>
```

Conclusion:

In this practical we learnt how we delete element and how we can print reverses element in linked list.

Key-Question Answer:

1 : [Problem 1] Inserting at a position beyond the current length of the list is an invalid operation. Should your solution silently insert at the end, report an error, or do something else entirely? Justify your choice and explain what assumption about the caller your choice makes.

-> The solution should report an error instead of silently inserting at the end. This makes the invalid position clear and prevents unexpected changes to the list. It assumes that the caller provides a valid position, but if not, the program safely handles the error.

2. [Problem 2] If the node to be deleted is the only node in the list, how many pointers need to change and what must they be set to? How is this case structurally different from deleting a node in the middle?

-> If the node is the only node, only one pointer (head) needs to change, and it must be set to NULL. In a middlenode deletion, the previous node's next pointer must be changed to skip the deleted node.

Practical 5:
Doubly Linked List and Circular Linked List Operations

Practical Aim-

To implement core doubly linked list and circular linked list operations and understand how bidirectional and circular linking change the way nodes are inserted, deleted, and traversed.

Problem 1 -

A music player maintains a playlist where songs can be added to the beginning, added to the end, or inserted right after a specific song that is currently playing. When a song is removed, only the first song in the playlist is dropped. At any point the player can count how many songs are in the playlist and display them from first to last. Given a sequence of such operations, implement all of them on a doubly linked playlist and print the result after each operation. Describe how your insertion after a given song works. What extra step does a doubly linked list require here that a singly linked list did not? What happens if the given song is not found in the playlist?

Code-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical%205/5_1.cpp

Output-

```
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB> cd "c:\Users
++ 5_1.cpp -o 5_1 } ; if ($?) { .\5_1 }
A
A B
A B C
A B X C
Count: 4
B X C
Count: 3
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB>
```

Conclusion

In this practical we learnt how we can add and delete element from front ,end and at a given specific posiition and how we can print elements in a doubly linked list.

Problem 2

A group of students are sitting in a circle playing a passing game. A token starts at the first student and is passed around the circle one student at a time. Students can join the circle at any position and leave at any time, and the circle must remain unbroken after every join or leave. Given a sequence of join, leave, and display operations, implement them on both a singly circular and a doubly circular linked list and print the current circle after each operation. Describe how your solution ensures the circle stays unbroken after a student leaves. What is the one case where breaking the circle is most likely to happen as a bug, and how did you handle it?

Code-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical%205/5_2.cpp

Output-

```
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB> cd "c:\Users\Pratham Gandh
++ 5_2.cpp -o 5_2 } ; if ($?) { .\5_2 }
10
join 10 1
join 20 2
join 30 3
display
Singly: 10 20 30
Doubly: 10 20 30
join 15 2
display
Singly: 10 15 20 30
Doubly: 10 15 20 30
leave 20
display
```

```
Singly: 10 15 30
Doubly: 10 15 30
leave 10
display
Singly: 15 30
Doubly: 15 30
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB>
```

Conclusion:

In this practical we learnt how we delete element and how we can print reverses element in a doubly linked list.

Key-Question Answer:

1 : [Problem 2] If your traversal of the circular list does not stop at the right moment, it will loop forever. Without referencing any specific code, explain what condition must be true at the moment you stop, and why that condition would fail if you set the last node's link to null.

-> In a circular linked list, traversal should stop when we reach the starting node again. This is because the last node points back to the first node.

If the last node's link is set to NULL, the list is no longer circular. The traversal would reach NULL instead of returning to the starting node, so the condition for a circular list would fail.

2. [Problem 1] A doubly linked list maintains two links per node instead of one. When deleting the front node, explain why having a backward link on the second node creates a responsibility that did not exist in a singly linked list — and what goes wrong if you forget to handle it.

-> In a doubly linked list, each node has both next and prev links. When we delete the first node, the second node becomes the new head. We must update the second node's prev to NULL

If we forget to update it, the second node's prev will still point to the deleted node. This creates an invalid link and can cause errors when moving backward through the list.

Practical 6:
Stack Implementation using Array and Linked List, with Expression Processing

Practical Aim-

To implement stack using array and linked list, and apply stack-based logic to expression conversion, evaluation, and bracket validation problems

Problem 1 -

A cafeteria stacks clean trays on a fixed-size counter. New trays are always placed on top, and customers always take from the top. The counter can hold at most n trays at a time — if it is full, no more trays can be added, and if it is empty, no tray can be taken. Given a sequence of place and take operations, implement this fixed-capacity tray stack and print the current top tray after each operation. Report an error if a place or take operation cannot be performed. Describe how your solution tracks whether the stack is full or empty. What are the exact conditions you check before a push and before a pop, and what does your program do when those conditions fail?

Code-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical%206/6_1.cpp

Output-

```
OneDrive\Desktop\DSA_LAB\" ; if ($?) { g++ 6_1.cpp -o 6_1 } ; if ($?
3
4
place 10
Top: 10
place 20
Top: 20
tale
take
Top: 10
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB>
```

Conclusion

In this practical, we learnt how to implement a stack using an array and perform place and take operations. We also learnt how to handle stack overflow and underflow conditions.

Problem 2

A web browser keeps track of pages visited so that the back button always returns to the most recently visited page. Unlike a fixed counter, the browser has no hard limit on how many pages it can remember — it grows as the user visits more pages and shrinks as they press back. Given a sequence of visit and back operations, implement this unlimited page history and print the current page after each operation. Describe how your solution differs from Problem 1 in how it handles memory. What happens in your solution when the user presses back on the very first page with no history left?

Code-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical%206/6_2.cpp

Output-

```
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB> cd "c:\Users\Pratha
OneDrive\Desktop\DSA_LAB\" ; if ($?) { g++ 6_2.cpp -o 6_2 } ; if ($?) {
3
visit google
google
visit github
github
back
google
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB>
```

Conclusion:
In this practical, we learnt how to implement a stack using a linked list and perform browser history operations such as visiting a new page and going back to the previous page.

Problem 3
A calculator application receives arithmetic expressions typed by users in the usual way — numbers and operators written in between, with brackets to control priority (e.g. 3 + 4 * 2 or (3 + 4) * 2). Internally the calculator needs to convert each expression into a form where operators appear after their operands, so it can evaluate them without needing to re-read brackets. Given an infix expression, convert it to the equivalent postfix form and print the result.

Code-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical%206/6_3.cpp

Output-

```
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB> c
OneDrive\Desktop\DSA_LAB\" ; if ($?) { g++ 6_3.cpp -o
a+b*(c-d)
abcd-*+
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB>
```

Conclusion:
The program converts an infix expression into postfix form using a stack and operator precedence.

Key-Question Answer:

Q1. A circular array is one way to reuse freed slots without shifting all elements. Without implementing it, explain what change would need to be made to your front and rear pointer logic to make the array behave circularly, and what new condition would indicate a full array.

Answer: Use modulo (%) to wrap the front and rear pointers back to index 0 after reaching the last index. The array is full when (rear + 1) % size == front.

Q2. Your linked list stack has no fixed capacity limit. But in practice, can it still run out of space? What determines the actual limit, and why is that limit invisible in your code?

Answer: Yes, it can run out of space when the system's available memory is exhausted. The limit depends on available memory and is not fixed in the code.

Q3. Operator precedence determines the order in which operators are placed into the output. Without tracing any expression, explain what would go wrong in your output if your solution treated all operators as having equal precedence.

Answer: The postfix expression could have an incorrect order of operations because operators like + and * would be treated equally, changing the intended evaluation order.

Practical 7:
Queue Implementation using Array and Linked List, with Structure Simulation

Practical Aim-

To implement queue using array and linked list, and apply queue-based logic to number generation, and structure simulation problems.

Problem 1 -

A government office has a token counter that issues tokens to visitors. The counter can hold at most n tokens at a time. New visitors join from one end and are served from the other end in the order they arrived. If the counter is full, no new token can be issued, and if it is empty, no one can be served. Given a sequence of join and serve operations, implement this fixed-capacity token system and print the current front token after each operation. Report an error if an operation cannot be performed. Describe how your solution tracks the front and rear of the queue. What happens to your front and rear pointers over time as join and serve operations keep happening, and does your solution handle the case where space appears to run out even though slots have been freed?

Code-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical%20%207/7_1.cpp

Output-

```
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB> cd "c:\Use
OneDrive\Desktop\DSA_LAB\" ; if ($?) { g++ 7_1.cpp -o 7_1 } ; i
Enter queue capacity: 5
Enter number of operations: 4

1. Join
2. Serve
Enter operation: 1
Enter token: 101
Front token: 101

1. Join
2. Serve
Enter operation: 1
Enter token: 102
Front token: 101

1. Join
```

```
Enter operation: 1
Enter token: 102
Front token: 101

1. Join
2. Serve
Enter operation: 2
Served token: 101
Front token: 102

1. Join
2. Serve
Enter operation: 1
Enter token: 1
Front token: 102
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB>
```

Conclusion

In this practical, we learnt how to insert and delete elements in a circular

queue, check whether the queue is full or empty, and reuse freed spaces using front and rear pointers.

Problem 2

A hospital emergency ward receives patients continuously and attends to them strictly in the order they arrived, with no upper limit on how many can be waiting at once. New patients are added at the back and the doctor always attends to the patient at the front. Given a sequence of arrive and attend operations, implement this unlimited patient queue and print the current front patient after each operation. Describe how your solution differs from Problem 1 in how it manages capacity. What happens in your solution when the attend operation is called on an empty ward with no patients waiting?

Code-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical%20%207/7_2.cpp

Output-

```
OneDrive\Desktop\DSA_LAB\" ; if ($?) { g++ 7_2.cpp -

1. Arrive
2. Attend
3. Display Front
4. Display Queue
5. Exit
Enter choice: 1
Enter patient number: 101
Patient added.
Front patient: 101

1. Arrive
2. Attend
3. Display Front
4. Display Queue
```



Conclusion:
In this practical, we learnt how to add and remove patients from a linked queue, display the front patient, and print all the elements in the queue.

Key-Question Answer:

Q1. A simple array queue eventually reports being full even when slots have been freed at the front. Explain why this happens structurally — not by tracing an example, but by describing what the front and rear pointers represent and what their movement pattern causes over time.

Answer: The front points to the first element, while the rear points to the last element. In a simple array queue, the rear moves only forward and cannot reuse freed slots at the front, so the queue may become full even when some slots are empty.

Q2. Your linked list queue has no capacity limit. If arrive operations vastly outnumber attend operations over a long period, what real consequence would this have on the system, and where in your code is that consequence invisible?

Answer: The queue keeps growing and consumes more memory, which may eventually exhaust the system's available memory. This limit is invisible because the code does not check available memory before creating new nodes with new.

Practical 8:
Binary Tree Traversals and Binary Search Tree Insertion

Practical Aim-

To implement binary tree traversals and binary search tree insertion, and develop the ability to reason about how the order of visiting nodes changes the information retrieved from a tree.

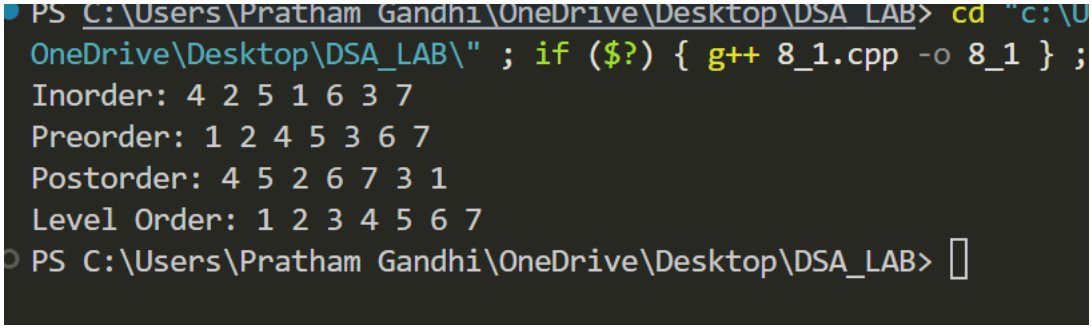
Problem 1 -

A company stores its organizational hierarchy as a tree — the CEO at the top, managers below, and employees at the leaves. Different departments need the hierarchy printed in different orders: HR wants each manager printed between their left and right sub-teams, the archive department wants each manager printed before their sub-teams, the payroll department wants each manager printed after all their sub-teams are listed, and the floor manager wants everyone printed level by level from top to bottom. Given a binary tree representing the hierarchy, implement all four printing orders and display the result of each. Describe how your solution for level-by-level printing differs from the other three. What additional structure did you need to use, and why does the same recursive approach that works for the other three not directly apply here?

Code-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical%208/8_1.cpp

Output-



Conclusion

In this practical, we learnt how to create a binary tree and display its elements using inorder, preorder, postorder, and level-order traversal.

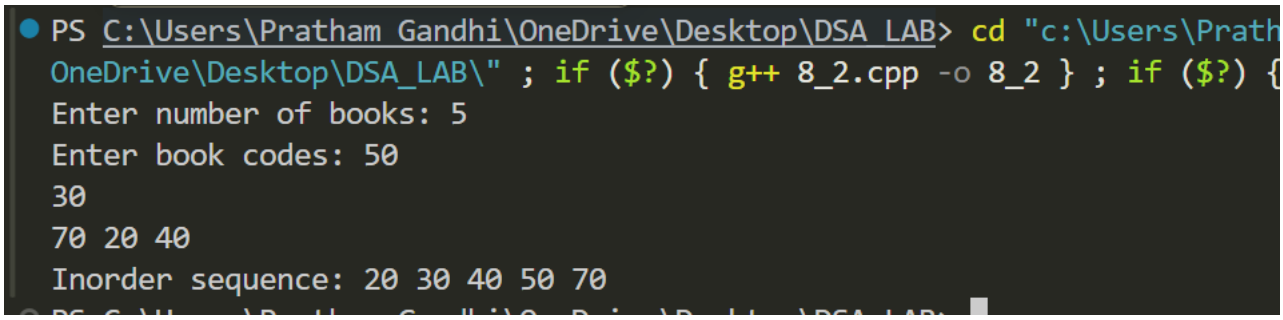
Problem 2

A school library uses a system where every book is assigned a unique code when it arrives. Books are organized so that at any shelf position, all books to the left have smaller codes and all books to the right have larger codes. This arrangement lets staff find any book quickly by always knowing which direction to look. Given a sequence of book codes arriving one by one, insert each into the correct position and print the inorder sequence after all insertions are complete. Describe how your solution decides where to place each new book code. What property of the final inorder sequence tells you whether all insertions were performed correctly?

Code-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical%208/8_2.cpp

Output-



Conclusion:

In this practical, we learnt how to insert book codes into a Binary Search Tree (BST) and display them in sorted order using inorder traversal.

Key-Question Answer:

1. Question: Preorder traversal always visits the root before anything else. Using only this property — without tracing any tree — explain whether it would be possible to reconstruct the original tree from its preorder output alone, and what additional information you would need if it is not.

Answer: No, preorder alone is not enough to reconstruct a unique tree. We also need inorder traversal or additional information about the tree's structure.

2. Question: The inorder traversal of a correctly built BST always produces a sorted sequence. If your inorder output is not sorted after all insertions, what does that tell you about where the error is — in the insertion logic, the traversal logic, or possibly both? How would you isolate which one?

Answer: If inorder output is not sorted, the error may be in insertion logic or traversal logic. Check whether the BST is built correctly, then test inorder traversal on a known tree to identify the mistake.

Practical 9:
Graph Traversal: Depth-First Search and Breadth-First Search

Practical Aim-
To implement depth-first and breadth-first traversal on a graph and develop the ability to reason about how the order of exploration changes depending on the traversal strategy used.

Problem 1 -
A disaster relief team needs to inspect all buildings in a city after an earthquake. The city is mapped as a network where buildings are connected by roads. One team starts at a building and goes as deep into each road as possible before backtracking, while a second team spreads out level by level — first checking all buildings directly connected to the start, then all buildings two roads away, and so on. Given the city map as a graph and a starting building, implement both exploration strategies and print the order in which buildings are visited by each team. Describe how the order of buildings visited differs between the two strategies on the same graph. What data structure does each strategy rely on to decide which building to visit next, and how does that structure cause the difference in order?

Code-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical%20%209/9_1.cpp

Output-

```
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB> cd "c:\Users\Pratha
OneDrive\Desktop\DSA_LAB\" ; if ($?) { g++ 9_1.cpp -o 9_1 } ; if ($?) {
5 4
0 1
0 2
1 3
1 4
0
DFS: 0 1 3 4 2
BFS: 0 1 2 3 4
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB>
```

Conclusion

In this practical, we learnt how to represent a graph using an adjacency list and traverse it using DFS and BFS.

Key-Question Answer:

1. Question: Both strategies visit every reachable node exactly once. If the goal is simply to check whether a specific building exists in the network, which strategy would find it sooner if the building is very close to the start? What if it is very far and deep? Explain your reasoning without tracing through any specific example.

Answer: BFS is usually faster for finding a nearby building because it explores level by level. DFS may find a far and deep building sooner because it explores deeper paths first, but it depends on the path order.

2. Question: Your DFS implementation likely uses recursion. What real-world constraint on the computer could cause your recursive DFS to fail on a very large city map, and how would you restructure your solution to avoid that failure?

Answer: A very large map may cause a stack overflow due to deep recursion. We can avoid this by using an explicit stack and implementing DFS iterative

Practical 10:
Hash Tables: Linear Probing, Chaining, and Double Hashing

Practical Aim-

To implement hash tables using different collision handling strategies and develop the ability to reason about why collisions occur, how different strategies resolve them, and what trade-offs each approach involves.

Problem 1 -

A small parking lot has exactly 10 numbered slots (0 to 9). When a vehicle arrives, its registration number is used to assign it a slot by taking the last digit of the number. If that slot is already taken, the vehicle moves to the next available slot in order, wrapping around if necessary. Given a sequence of vehicle registrations, assign each to a slot using this rule and display the final state of all ten slots. Describe what your solution does when it encounters an already occupied slot. What happens when the parking lot is nearly full and many vehicles keep landing on the same slot? Can you think of a situation where your solution might loop indefinitely?

Code-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical%2010/10_1.cpp

Output-

```
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB> cd "c:\Users
OneDrive\Desktop\DSA_LAB\" ; if ($?) { g++ 10_1.cpp -o 10_1 } ; i
}
5
12 22
15 25 32
Parking slots: -1 -1 12 22 32 15 25 -1 -1 -1
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB>
```

Conclusion

In this practical, we learnt how to store values in a hash table using the modulo method and linear probing to handle collisions.

Problem 2

A library uses a system with 10 shelves numbered 0 to 9. Each book's code is used to decide which shelf it belongs to by taking the code modulo 10. Unlike the parking lot, each shelf can hold multiple books stacked together — when two books map to the same shelf, they are simply added to that shelf's stack. Given a sequence of book codes, place each on the correct shelf and display the final contents of all shelves. Describe how your solution handles two books that map to the same shelf. What happens to search time on a shelf as more and more books pile onto it? Can you think of what would cause all books to end up on the same shelf?

Code-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical%2010/10_2.cpp

Output-

```
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB> cd "c:\Users\Pratham G
OneDrive\Desktop\DSA_LAB\" ; if ($?) { g++ 10_2.cpp -o 10_2 } ; if ($?) { .
}
5
12 22 15 25 32
0:
1:
2: 12 22 32
3:
4:
5: 15 25
6:
7:
8:
9:
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB>
```

Conclusion:
In this practical, we learnt how to handle hash table collisions using separate chaining by storing values with the same hash index in a list.

Problem 3
A university assigns student IDs to storage slots using a two-step rule. The first rule determines the initial slot. If that slot is taken, instead of simply moving one slot forward, a second rule based on the ID itself determines how many slots to jump. This means different IDs that collide will probe different sequences of slots, spreading them more evenly. Given a sequence of student IDs, assign each to a slot using both rules and display the final state of the table.
Describe how your solution decides how many slots to jump when a collision occurs. What is the risk if the jump size happens to be a factor of the table size? How does this strategy differ from Problem 10.1 in how it spreads collisions?
Code-

https://github.com/PrathamGandhiIT/FDSA_ARP_BATCHA/blob/main/DSA_Lab/Practical%2010/10_3.cpp

Output-

```
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB> cd "c:\Users\Pratham G
OneDrive\Desktop\DSA_LAB\" ; if ($?) { g++ 10_3.cpp -o 10_3 } ; if ($?) { .
}
5
12 22 15  25 32
Hash table: -1 25 12 -1 32 15 -1 -1 22 -1
PS C:\Users\Pratham Gandhi\OneDrive\Desktop\DSA_LAB>
```

Conclusion:
In this practical, we learnt how to handle hash table collisions using double hashing with two hash functions to find suitable positions.

Key-Question Answer:

1. Question: If five consecutive vehicle registrations all end in the same digit, trace where each one gets placed in the parking lot. Now explain why this clustering effect is a structural weakness of this collision strategy — not just for this input, but in general.

Answer: If all five registrations have the same last digit, d, they occupy slots d, (d+1)%10, (d+2)%10, (d+3)%10, and (d+4)%10, assuming these slots are initially empty. This causes clustering because consecutive slots become occupied, making future insertions slower.

2. Question: Suppose all book codes in Problem 2 are multiples of 10. Without tracing through the code, predict what the final shelf layout would look like and

explain why. What does this tell you about the relationship between the hash function and the input data?

Answer: All book codes would have the same hash index because each is a multiple of 10. They would be stored together using separate chaining. This shows that the hash function should distribute the input data evenly.

3. Question: In Problem 3, what would happen if the second rule always produced a jump size of zero? Trace through the consequence and explain how this failure is related to the choice of the second hash function rather than the data itself.

Answer: If the jump size is zero, every attempt checks the same slot. If that slot is occupied, the loop checks it ten times and then reports the table as full, even if other slots are empty. This happens because the second hash function produces an invalid jump size.
